KLU+PATH-3 · ML-02 HACKATHON

Catch fraud, not customers

A cost-aware Approve / Step-up OTP / Block system for UPI and digital payments

Team: [names]

github.com/MohammedMutee/upi-fraud-detection

THE ACCURACY TRAP

A 99.87%-accurate model that catches zero fraud

99.87%

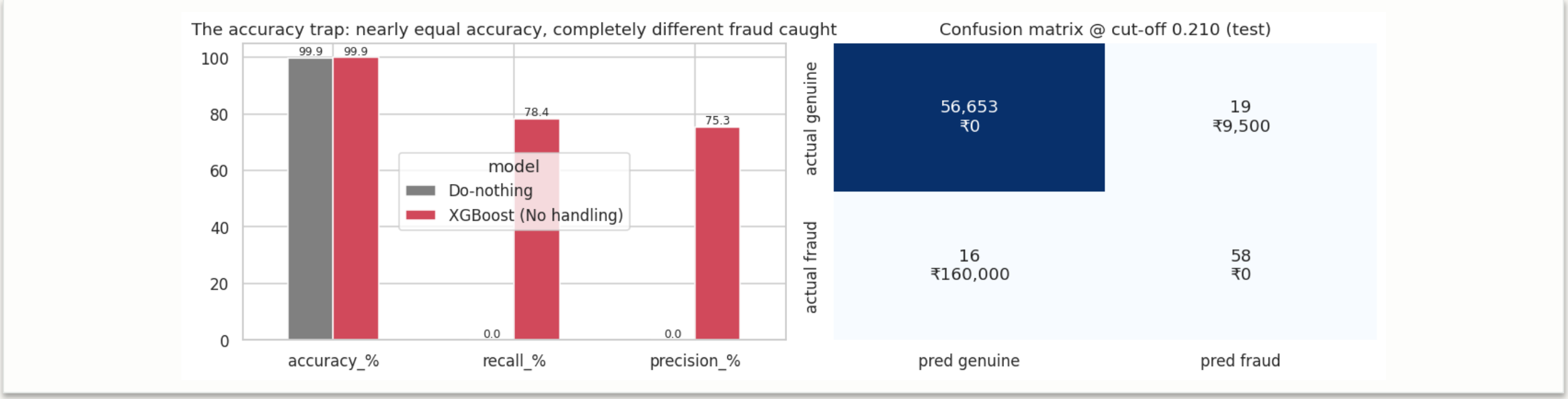
accuracy of a model that says “genuine” to everything

0 of 74

frauds it catches on the test set

78%

caught by our model, at almost the same accuracy



Card / digital channel, test slice (newest 20% of transactions). We use recall, precision, PR-AUC and expected ₹ cost instead.

THE BUSINESS PROBLEM

Every decision has two ways to lose money

Miss a fraud

Money lost and trust lost.
Cost assumed: ₹10,000 each

Block a good user

Angry customer, may switch apps.
Cost assumed: ₹500 each

Ask for an OTP

5 seconds of friction.
Cost assumed: ₹10 each

Channel	Dataset	Payments	Frauds
UPI	upi_transactions 2024.csv (simulated, per the brief)	250,000	480 (0.19%)
Card / digital	creditcard.csv (real, ULB)	283,726 (after cleaning)	473 (0.17%)

⚠️ The ₹ costs are placeholder assumptions (no official values were published). Every saving shown is a scenario estimate.

PS1 · DATA QUALITY

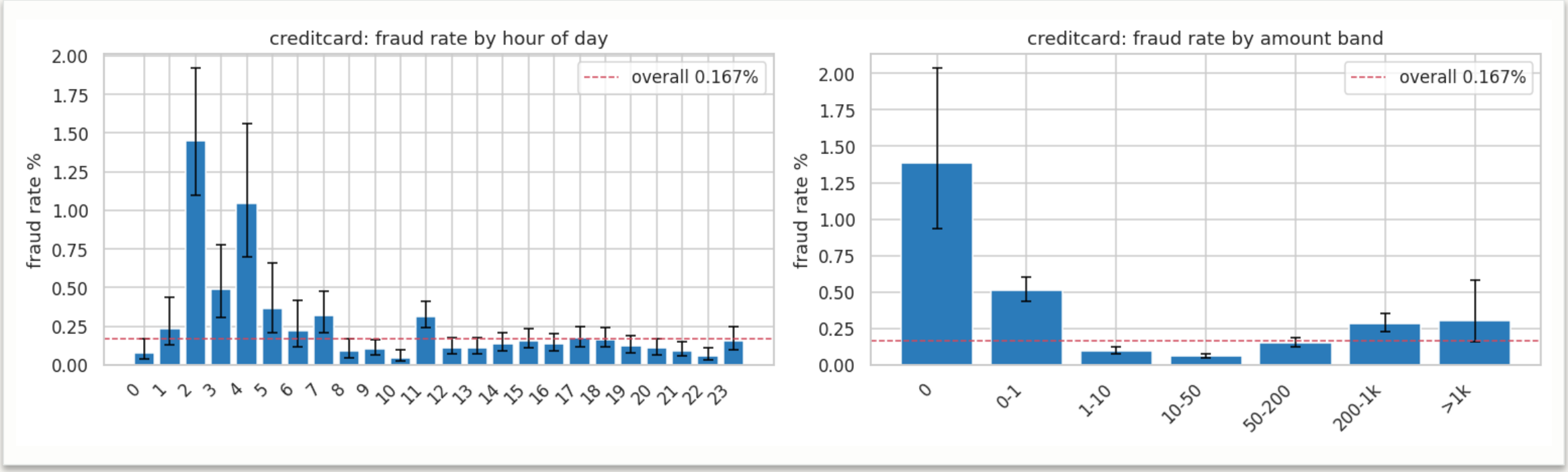
Clean the data, and remove the columns that cheat

Check	Found	What we did
Exact duplicate rows (card)	1,081 (19 fraud)	Dropped: a copy in train and test inflates scores
Amount = 0 (card)	1,825	Kept: likely card-verification pings
Missing values / bad timestamps	0 in both files	No action
UPI duplicates / bad categories	0: “too perfect” for real data	Noted: first hint the UPI file is synthetic
transaction_status (UPI)	12,376 FAILED	EXCLUDED: only known after processing (leakage)
transaction id (UPI)	250,000 unique	EXCLUDED: an identifier, not a signal

Split: time-aware. Oldest 60% train · next 20% validation (every choice made here) · newest 20% test (reported once).

PS3 · WHERE FRAUD LIVES

Fraud peaks at night and on tiny “test” payments



1–5 AM: 0.67% fraud vs 0.14% at other hours (about 5×), peaking at 1.45% at 2 AM.

Amount is U-shaped: ₹0–1 payments 0.51% and mid amounts 0.06%. UPI: every type, bank and hour sits on the overall rate.

Error bars = 95% confidence intervals. Observed associations only; no causal claims.

PS4 · SUPERVISED MODELS

Nine models against a hand-written rule

Model	PR-AUC	Frauds caught	Genuine blocked	Test cost
Rule: night + tiny amount	0.002	28 / 74	6,343	₹36,31,500
Logistic Regression (Class weights)	0.745	55 / 74	24	₹2,02,000
Random Forest (Class weights)	0.819	56 / 74	9	₹1,84,500
Random Forest (SMOTE)	0.812	61 / 74	81	₹1,70,500
XGBoost (No handling)	**0.796**	**58 / 74**	**19**	**₹1,69,500**
XGBoost (Class weights)	0.791	56 / 74	10	₹1,85,000

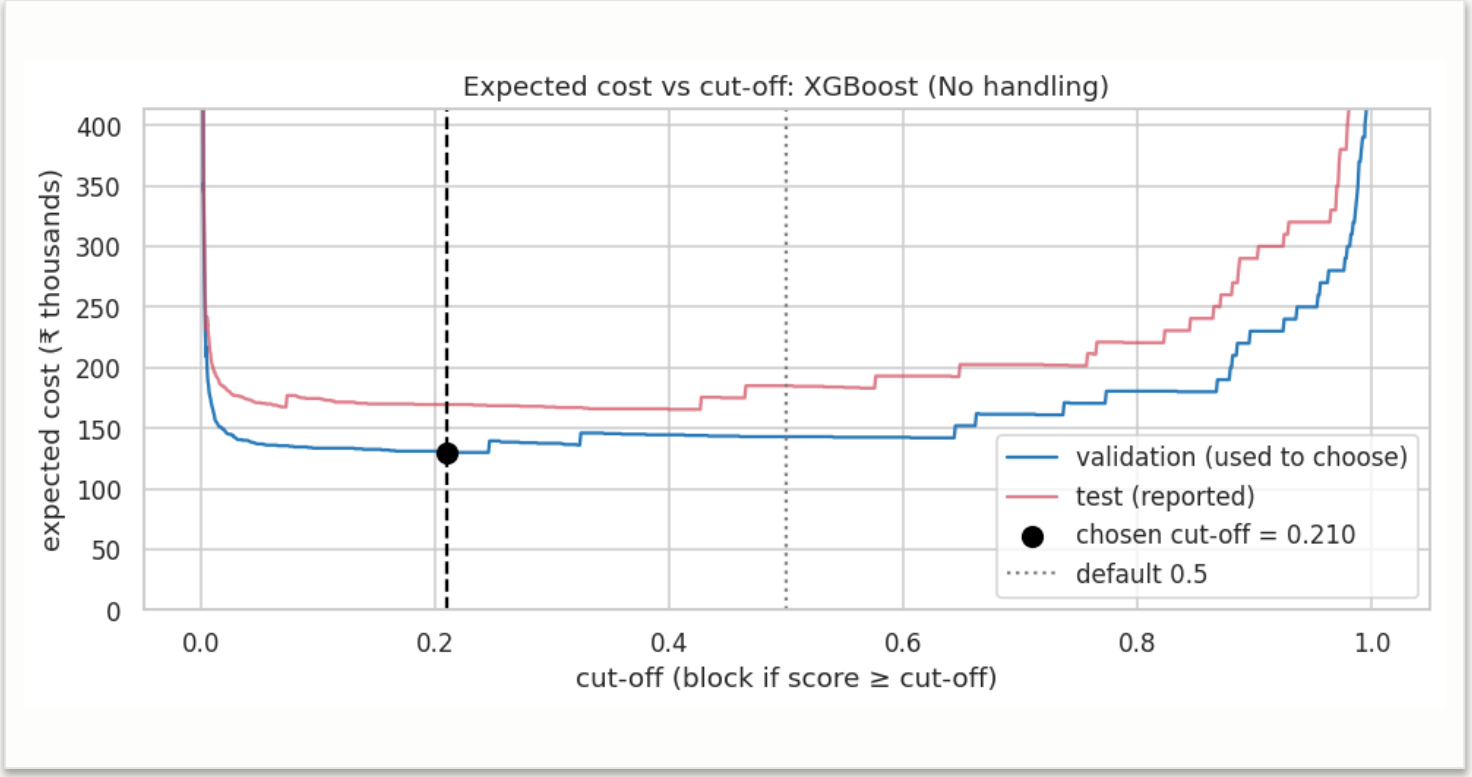
Rules alone fail: the rule costs about 5× more than doing nothing.

Imbalance tricks help simple models (LR: 0.61 → 0.75 PR-AUC). For trees, a cost-tuned cut-off does the same job.

Card / digital channel, test slice. SMOTE applied only inside the training pipeline. Highlighted = final model (chosen on validation).

PS6 · MONEY PICKS THE CUT-OFF

The cheapest line is 0.21, not 0.5



₹7.40 lakh

do nothing (approve all)

₹1.85 lakh

default 0.5 cut-off

₹1.70 lakh

cost-chosen 0.21 cut-off

At 0.21: 58 of 74 frauds caught, only 19 genuine customers blocked.

Test-slice expected cost under placeholder costs (scenario estimate). The cut-off was chosen on validation, never on test.

PS7 · THE DECISION RULE

Approve 96.6% · OTP 3.2% · Block 0.1%

Approve

96.6% of payments

10 frauds slip through

Step-up OTP

3.2% of payments

8 frauds · 1,828 genuine asked

Block

0.1% of payments

56 frauds (76%) · only 9 genuine

₹1.39 lakh

3-zone rule, test cost

81%

cheaper than doing nothing

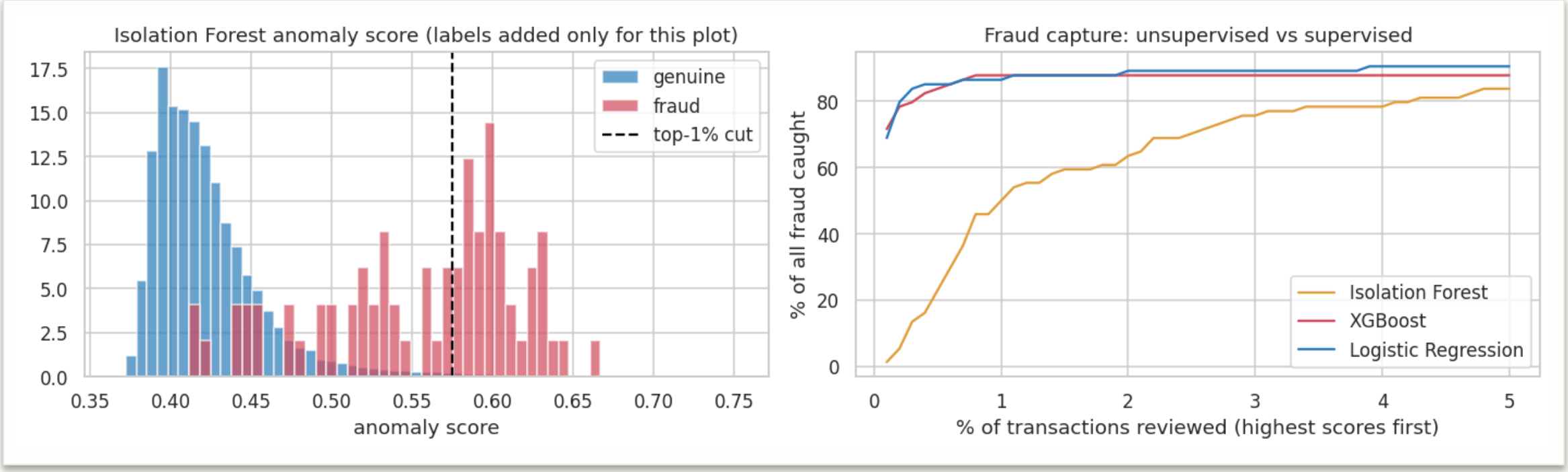
18%

cheaper than the best single cut-off

OTP assumed to stop 80% of fraud; OTP zone capped at 5% of traffic. Scenario estimate under placeholder costs.

PS5 · UNKNOWN FRAUD

A detector that never saw a label finds half the fraud



50%

of fraud in its top 1%, with no labels

84%

of fraud in its top 5%

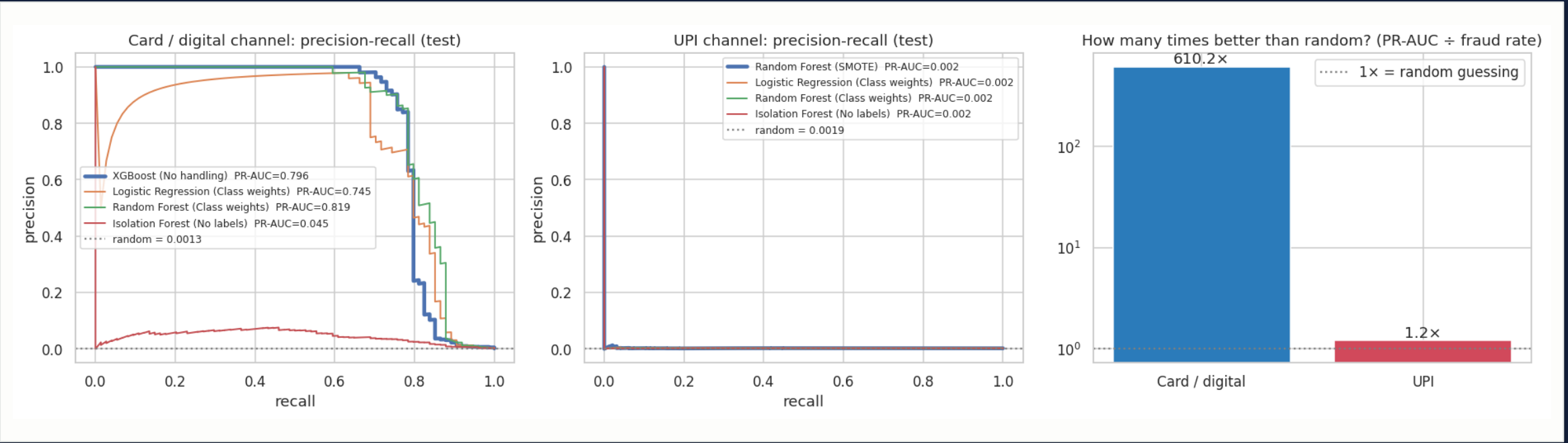
0.95 vs 0.05

ROC-AUC vs PR-AUC: why ROC flatters

Isolation Forest: 300 trees, trained on features only; labels used afterwards to evaluate.

TWO CHANNELS, ONE PIPELINE

Cards: 610× better than random. UPI: 1.2×



78% caught · 81% saved

Card / digital: real labels, real signal

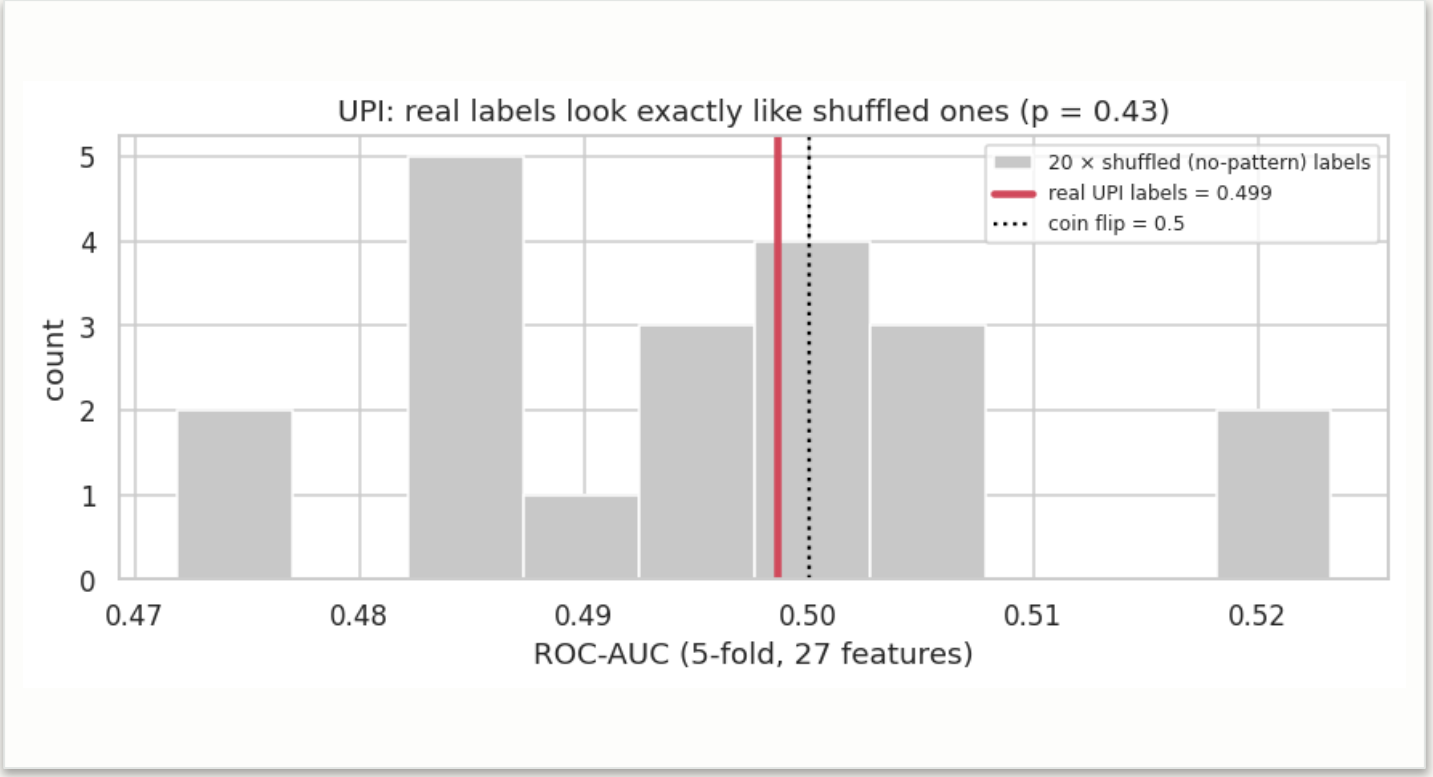
0% caught · 2 blocked

UPI: of 50,000 payments. No signal, so it refuses to block

PR-AUC ÷ fraud rate on each test slice. Identical code, no channel-specific tuning.

WHY ARE UPI RESULTS LOW?

Real UPI labels score the same as shuffled ones



0.499

ROC-AUC with the real UPI labels

0.495 ± 0.013

20 × randomly shuffled labels

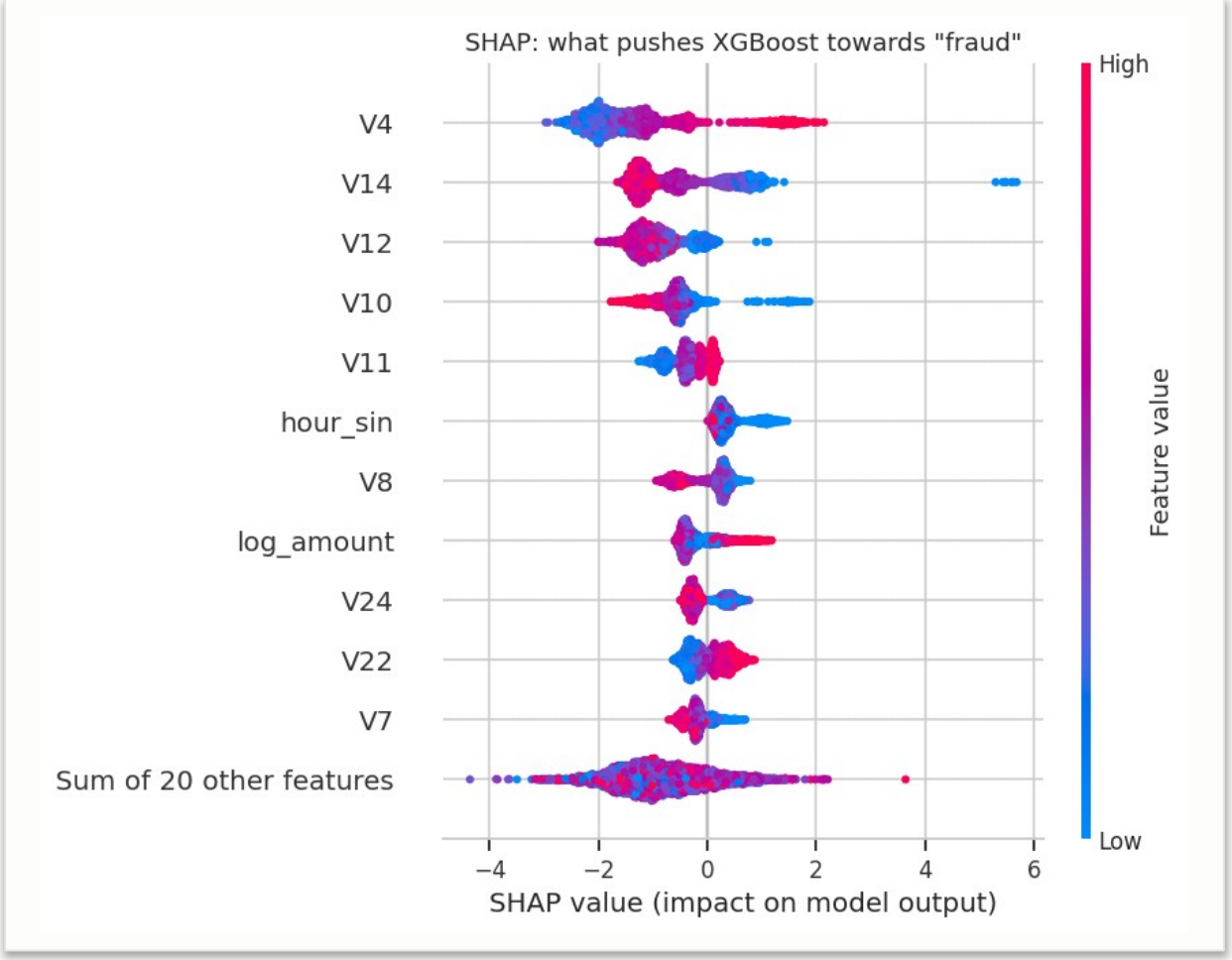
p = 0.43

no evidence of any pattern

Cleaning fixes broken values, and this file has none. The labels don’t depend on the inputs: a labelling problem, not a model problem.

27 features (14 engineered), gradient boosting, 5-fold cross-validation. Script: scripts/upi_rescue_test.py

Explaining every decision with SHAP



Top drivers: V4, V14, V12, V10, V11

Also matter: hour of day and amount, consistent with the EDA

Read it: red = high feature value; right of centre = pushes towards fraud

Limit: V1–V28 are anonymised (PCA), so we rank them but cannot name them

SHAP TreeExplainer on 3,000 test payments, final XGBoost model.

PS8 · THREE RECOMMENDATIONS

Finding → evidence → action

1 · Pilot the 3-zone rule

81% cheaper than doing nothing; blocks only 0.1% of payments

Run XGBoost + Approve/OTP/Block on a slice of live card traffic. Re-tune the cut-offs monthly with real costs.

2 · Add a no-label safety net

Isolation Forest: 50% of fraud in its top 1%, without labels

Send unusual-but-approved payments to a human review queue to catch new scam types early.

3 · Fix UPI fraud labelling

UPI labels: 1.2× random, the same as shuffled labels (p = 0.43)

Label from confirmed complaints and chargebacks. Until then: approve + capped OTP on UPI, never auto-block.

Priority: 1 and 3 high, 2 medium. Observed results and projected savings are kept separate.

LIMITATIONS

What is measured, what is assumed

Limitation	Effect
₹ costs are placeholders	Savings are scenario estimates; cut-offs may move (stable for cost ratios 10–100)
OTP stop rate (80%) and 5% OTP cap	Assumptions: they change the OTP zone, not the model
Card data covers 2 days; 74 test frauds	One fraud = 1.4% recall; no seasonal patterns
V1–V28 anonymised	We can rank drivers but not name them
UPI data is simulated	Its labels carry no signal; UPI results show behaviour, not patterns
Observational data	Associations, never causes

Re-run with official costs: change one cell in the notebook (about 15 min end to end).

Catch fraud, not customers

- Accuracy lies: 99.87% accurate, 0 frauds caught
- Cost-chosen 3-zone rule: 78% of fraud caught, 81% cheaper than doing nothing
- Same pipeline on UPI: no signal in the labels, so it safely refuses to block

Questions?

github.com/MohammedMutee/upi-fraud-detection